



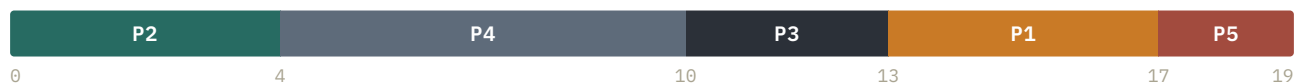
Priority Scheduling

Non-Preemptive vs Preemptive — priority-based CPU scheduling compared on a shared workload

NON-PREEMPTIVE EXECUTION



PREEMPTIVE EXECUTION



SAME FIVE PROCESSES, SAME PRIORITIES — P3 AND P4 SWAP ORDER, EVERYTHING ELSE LINES UP

5

PROCESSES

19

MAKESPAN (UNITS)

NP

LOWER AVG. WAIT

0.60

WT GAP (UNITS)

COURSE

CSE 402 — Operating System Lab

EXPERIMENT

Priority Scheduling — Non-Preemptive & Preemptive

REPOSITORY

github.com/ashadsetu/CSE-402

SUBMITTED BY

MD Asadullah

Reg. No 23101126 · Section C1
4th Year, 1st Semester

SUBMITTED TO

Atia Rahman Orthi

Lecturer, Department of CSE
University of Asia Pacific

PROBLEM

Schedule five processes by priority in two modes. In **non-preemptive** mode, once a process starts it runs to completion. In **preemptive** mode, a running process is interrupted the instant a better-priority process arrives. Lower priority number means higher priority in both.

PID	AT	BT	PR
P1	3	4	2
P2	0	4	0
P3	2	3	1
P4	1	6	1
P5	4	2	3

Both scripts run on this exact same set, so any difference in the results comes from the scheduling policy alone — not from different input.

NON-PREEMPTIVE

CODE — PRIORITY (NON-PREEMPTIVE)

```
pid = ["P1", "P2", "P3", "P4", "P5"]
AT = [3, 0, 2, 1, 4]
BT = [4, 4, 3, 6, 2]
PR = [2, 0, 1, 1, 3]
n = 5

completed = [False] * n
CT = [0] * n
TAT = [0] * n
WT = [0] * n

time = 0
finished = 0
sequence = []

while finished < n:
    idx = -1
    min_priority = 9999

    for i in range(n):
        if not completed[i] and AT[i] <= time:
            if PR[i] < min_priority:
                min_priority = PR[i]
                idx = i

    if idx == -1:
        time += 1
    else:
        time += BT[idx]
        CT[idx] = time
        completed[idx] = True
        finished += 1
        sequence.append(pid[idx])

for i in range(n):
    TAT[i] = CT[i] - AT[i]
    WT[i] = TAT[i] - BT[i]

avg_wt = sum(WT) / n
avg_tat = sum(TAT) / n

print("Process\tAT\tBT\tPR\tCT\tTAT\tWT")
for i in range(n):
    print(f"{pid[i]}\t{AT[i]}\t{BT[i]}\t{PR[i]}\t{CT[i]}\t{TAT[i]}\t{WT[i]}")

print("\nAverage Waiting Time =", avg_wt)
print("Average Turnaround Time =", avg_tat)
print("\nExecution Sequence:")
print(" -> ".join(sequence))
```

■ RESULT — NON-PREEMPTIVE

PID	AT	BT	PR	CT	TAT	WT
P1	3	4	2	17	14	10
P2	0	4	0	4	4	0
P3	2	3	1	7	5	2
P4	1	6	1	13	12	6
P5	4	2	3	19	15	13

```
Sequence: P2 -> P3 -> P4 -> P1 -> P5
Average TAT = 10.0      Average WT = 6.2
```

PREEMPTIVE

Same processes, same priorities — this version re-checks every time unit, so a better-priority arrival can cut in immediately.

■ CODE — PRIORITY (PREEMPTIVE)

```
processes = ["P1", "P2", "P3", "P4", "P5"]
arrival_time = [3, 0, 2, 1, 4]
burst_time = [4, 4, 3, 6, 2]
priority = [2, 0, 1, 1, 3]
n = len(processes)

remaining_time = burst_time.copy()
completion_time = [0] * n
waiting_time = [0] * n
turnaround_time = [0] * n
done = [False] * n

time = 0
completed = 0
gantt = []

while completed < n:
    idx = -1
    highest_priority = 9999

    for i in range(n):
        if arrival_time[i] <= time and not done[i] and remaining_time[i] > 0:
            if priority[i] < highest_priority or (priority[i] == highest_priority
            and (idx == -1 or arrival_time[i] < arrival_time[idx])):
                highest_priority = priority[i]
                idx = i

    if idx == -1:
        time += 1
        continue

    if not gantt or gantt[-1][0] != processes[idx]:
        gantt.append([processes[idx], time, time + 1])
    else:
        gantt[-1][2] = time + 1

    remaining_time[idx] -= 1
    time += 1

    if remaining_time[idx] == 0:
        done[idx] = True
        completed += 1
        completion_time[idx] = time
        turnaround_time[idx] = completion_time[idx] - arrival_time[idx]
        waiting_time[idx] = turnaround_time[idx] - burst_time[idx]

avg_wt = sum(waiting_time) / n
avg_tat = sum(turnaround_time) / n

print("P\tAT\tBT\tPR\tCT\tTAT\tWT")
for i in range(n):
    print(f"{processes[i]}\t{arrival_time[i]}\t{burst_time[i]}\t{priority[i]}\t"
          f"{completion_time[i]}\t{turnaround_time[i]}\t{waiting_time[i]}")

print(f"\nAverage Waiting Time = {avg_wt:.2f}")
print(f"Average Turnaround Time = {avg_tat:.2f}")
print("\nGantt Chart:")
for g in gantt:
    print(f"{g[0]}({g[1]}-{g[2]})", end=" ")
```

■ RESULT — PREEMPTIVE

PID	AT	BT	PR	CT	TAT	WT
P1	3	4	2	17	14	10
P2	0	4	0	4	4	0
P3	2	3	1	13	11	8
P4	1	6	1	10	9	3
P5	4	2	3	19	15	13

Gantt Chart: P2(0-4) -> P4(4-10) -> P3(10-13) -> P1(13-17) -> P5(17-19)
 Average TAT = 10.60 Average WT = 6.80

VERDICT

Non-Preemptive

Avg TAT 10.0 • Avg WT 6.2

Preemptive

Avg TAT 10.6 • Avg WT 6.8

Non-Preemptive wins on both averages here — lower TAT (10.0 vs 10.6) and lower WT (6.2 vs 6.8), a 0.6-unit gap on each.

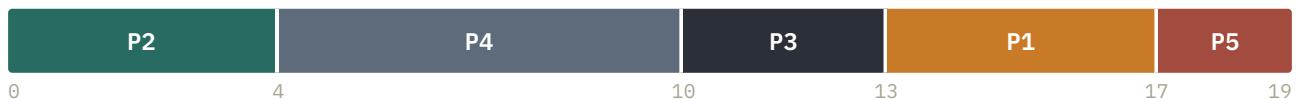
GANTT CHART

Drawn to scale — P2, P1, and P5 land identically in both; only P3 and P4 swap order.

NON - PREEMPTIVE



PREEMPTIVE



DISCUSSION

- **P3 and P4 tie on priority, and each script breaks the tie differently.** Both have priority 1 at time 2. The preemptive code explicitly tie-breaks by earliest arrival (`arrival_time[i] < arrival_time[idx]`), so P4 (arrived at t=1) wins and runs first. The non-preemptive code has no such rule — it just keeps the first match from a strict `<` scan, so P3 (earlier in the array, index 2) wins even though P4 arrived earlier. Same tie, two different — both defensible — outcomes.
- **That one tie-break is the entire source of the gap.** P1, P2, and P5 land on identical start and end times in both runs. Only P3 and P4 swap order, and that swap alone accounts for the full 0.6-unit difference in both average TAT and WT.
- **Preemption didn't help here — it cost a little.** Letting P4 (the longer job, burst 6) jump in early under preemptive mode pushes P3 later, and P3 (burst 3, priority same as P4) ends up waiting 8 units instead of 2. Preemption is meant to improve responsiveness for urgent arrivals, not average wait — this run shows that trade-off directly.
- **Both modes still let P5 starve toward the end.** Priority 3 is the lowest in the set, and P5 finishes dead last (CT 19) in both runs regardless of mode. Preemption changes who runs when among the higher-priority processes, but it doesn't fix the structural risk that a low-priority process wait indefinitely if higher-priority work keeps arriving — the standard fix is aging, raising priority the longer a process waits.

CONCLUSION

- On this shared workload, **Non-Preemptive Priority Scheduling wins narrowly** — Avg WT 6.2 vs 6.8, Avg TAT 10.0 vs 10.6 — entirely because of how each script resolves the P3/P4 tie, not because non-preemptive scheduling is inherently better.
- **Preemption trades average wait for responsiveness.** It lets a process react the instant something better-priority shows up, at the cost of interrupting whatever's already running — useful when reaction time matters more than throughput.
- Neither mode solves starvation on its own; a low-priority process can still wait indefinitely under either policy without aging.

Non-Preemptive edges out Preemptive by 0.6 units on both TAT and WT here — but the real story is one tie-break rule, not a fundamental advantage of either mode.